

Rapport de laboratoire

Analyse dynamique Android avec MobSF & DIVA

Environnement : Windows 11 + Android Studio + Docker Desktop

Auteur : EZBIRI Amira

Cible : DIVA APK (Damn Insecure and Vulnerable Android App)

Ce rapport présente la mise en place d'un environnement d'analyse dynamique Android avec MobSF, puis son utilisation sur une application volontairement vulnérable. L'objectif est de documenter clairement la préparation de l'émulateur, le déploiement de MobSF, l'analyse de l'APK DIVA et les observations principales obtenues pendant l'exécution.

1. Objectifs du laboratoire

Le laboratoire vise à maîtriser l'analyse dynamique d'une application Android dans un cadre contrôlé. Plus précisément, il permet de :

- créer un AVD sans Play Store afin de limiter le bruit réseau et les conflits avec les services Google ;
- déployer MobSF via Docker et le relier à l'émulateur ;
- lancer l'analyse dynamique de DIVA ;
- observer les comportements runtime, les logs Android et les échanges HTTPS ;
- identifier plusieurs catégories de vulnérabilités mobiles.

2. Environnement utilisé

Élément	Détail
Système hôte	Windows 11
IDE / SDK	Android Studio avec Android SDK Platform-Tools
Conteneurisation	Docker Desktop
Émulateur	AVD x86_64 sans Play Store, API 29/30
Outil d'analyse	MobSF en mode dynamique
Application testée	DIVA APK

Le choix d'une image Android sans Play Store permet d'obtenir un environnement plus propre pour l'analyse. L'émulateur est également mieux adapté aux manipulations de proxy, de logs et de Frida.

3. Préparation de l'AVD

La première étape consiste à créer un appareil virtuel Android depuis Android Studio. Le profil recommandé est de type Pixel, avec une image système x86_64 Android API 29 ou 30 sans mention "Google Play".

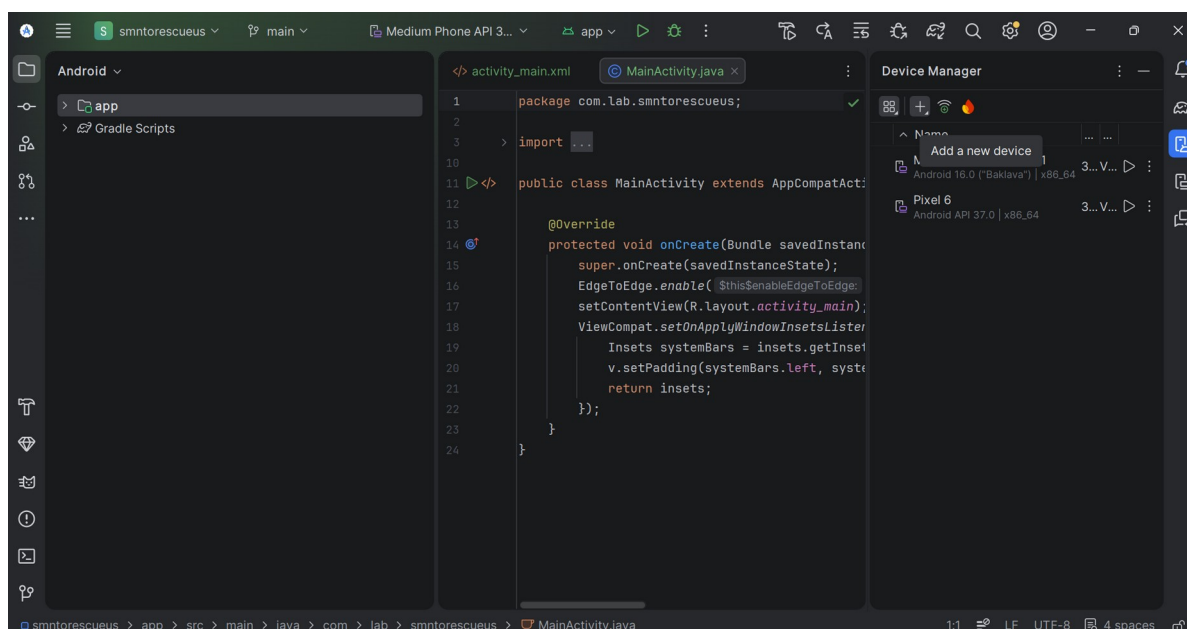


Figure 1 - Sélection du profil matériel dans Android Studio.

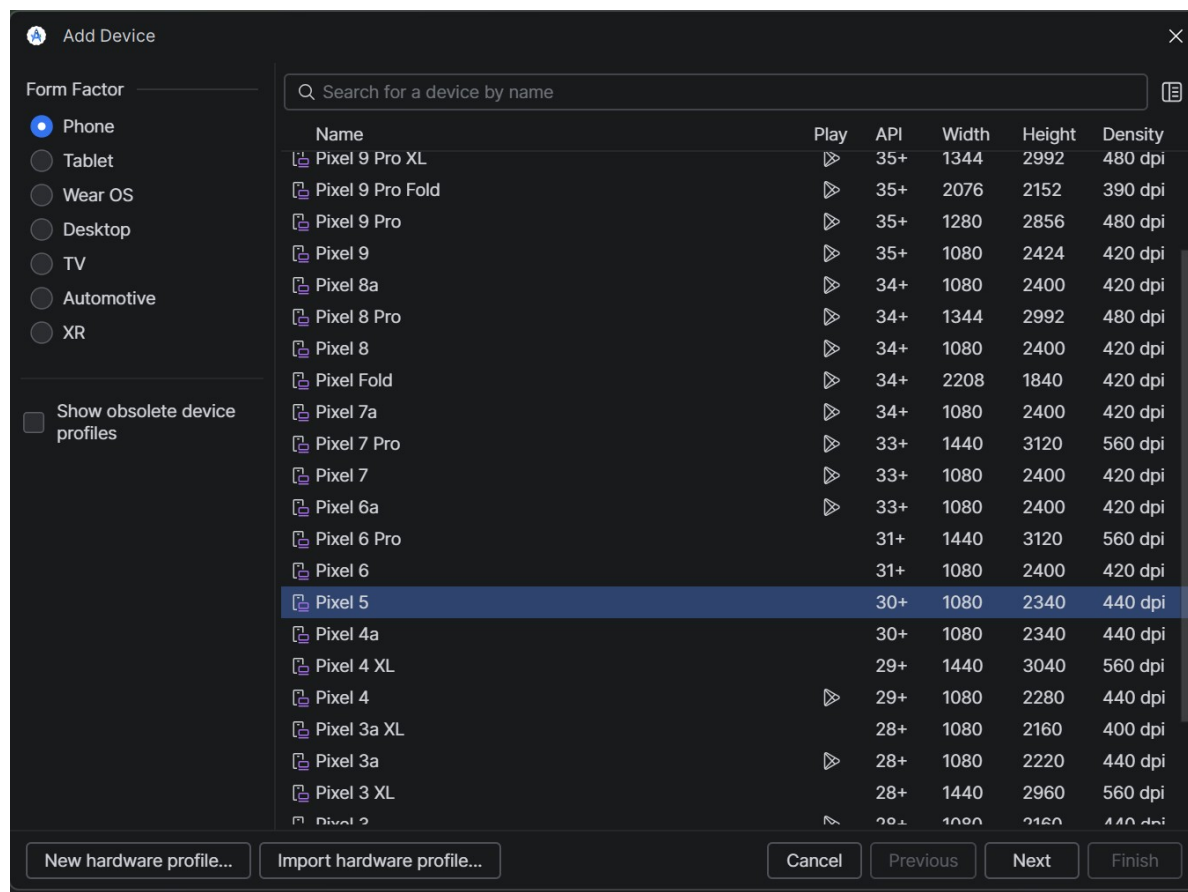


Figure 2 - Choix de l'image système Android sans Play Store.

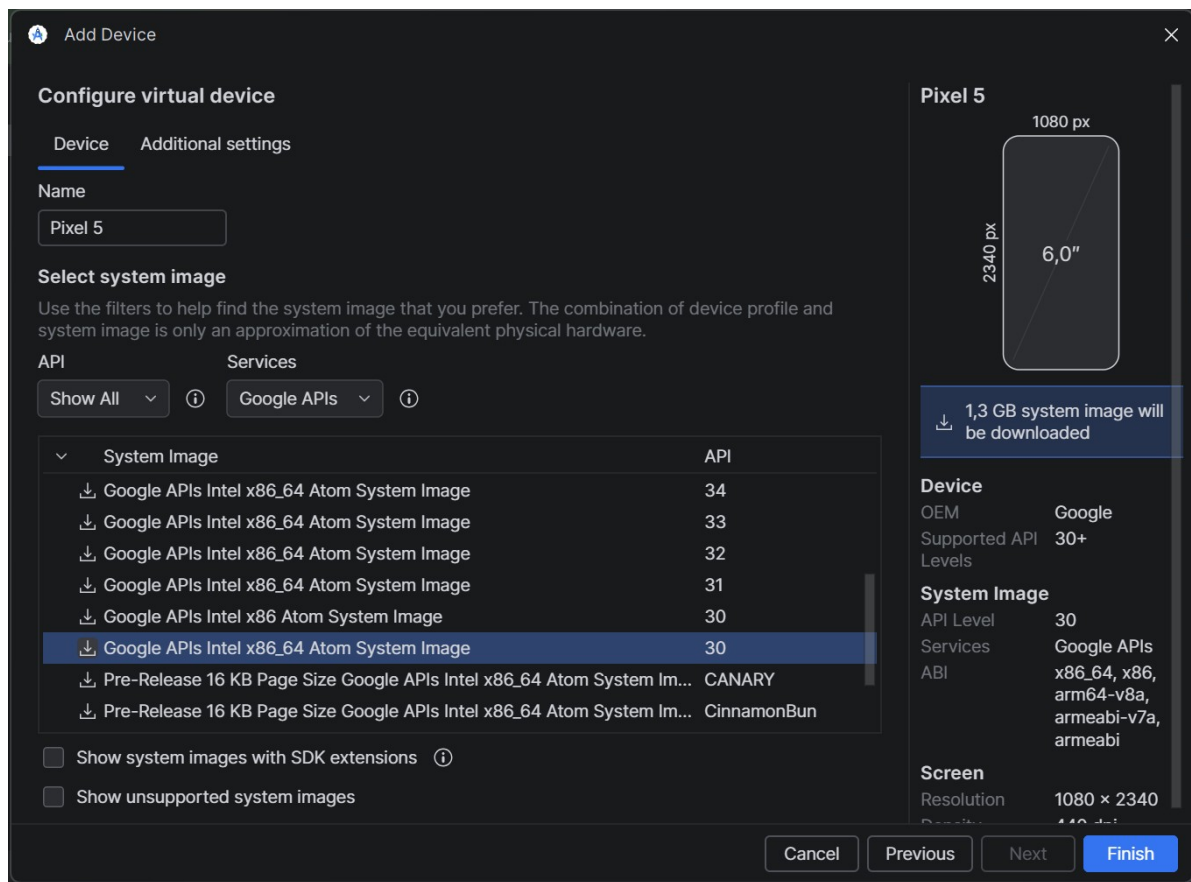


Figure 3 - Configuration finale de l'AVD.

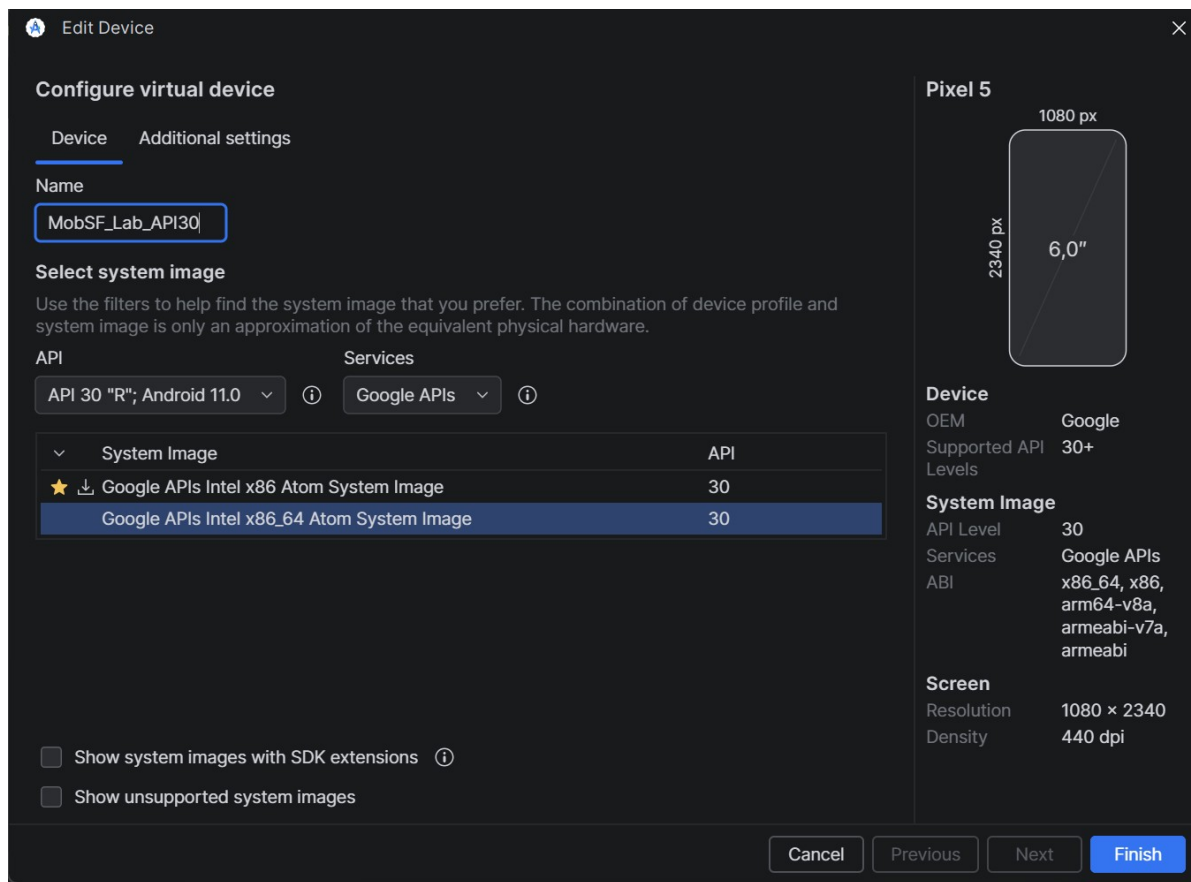


Figure 4 - Vérification de l'AVD créé.

À ce stade, l'AVD est prêt pour l'analyse et ne présente pas l'icône Play Store, ce qui confirme le bon choix de l'image système.

4. Installation et lancement de MobSF

Le dépôt officiel MobSF est d'abord cloné localement, puis le script de démarrage de l'émulateur est lancé. L'étape suivante consiste à vérifier la présence de l'émulateur avec ADB avant de démarrer MobSF dans Docker.

```
PS C:\WINDOWS\system32> git clone https://github.com/MobSF/Mobile-Security-Framework-MobSF.git
Cloning into 'Mobile-Security-Framework-MobSF'...
remote: Enumerating objects: 22245, done.
remote: Counting objects: 100% (58/58), done.
remote: Compressing objects: 100% (52/52), done.
remote: Total 22245 (delta 20), reused 6 (delta 6), pack-reused 22187 (from 3)
Receiving objects: 100% (22245/22245), 1.45 GiB | 1.35 MiB/s, done.
Resolving deltas: 100% (11937/11937), done.
Updating files: 100% (530/530), done.
PS C:\WINDOWS\system32>
```

Figure 5 - Clonage du dépôt MobSF.

```

PS C:\WINDOWS\system32> cd Mobile-Security-Framework-MobSF
PS C:\WINDOWS\system32\Mobile-Security-Framework-MobSF> .\scripts\start_avd.ps1
Available AVDs:
Medium_Phone_API_36.1
MobSF_Lab_API30
Pixel_6
Use any Android AVD 5.0 - 11, up to API 30 without Google Play (production image).
Usage: .\script.ps1 -AVD_NAME <AVD_NAME> [-START_PORT <START_PORT>]
Example: .\script.ps1 -AVD_NAME Pixel_6_Pro_API_28 -START_PORT 5554

```

Figure 6 - Présence du dépôt et du dossier de travail MobSF.

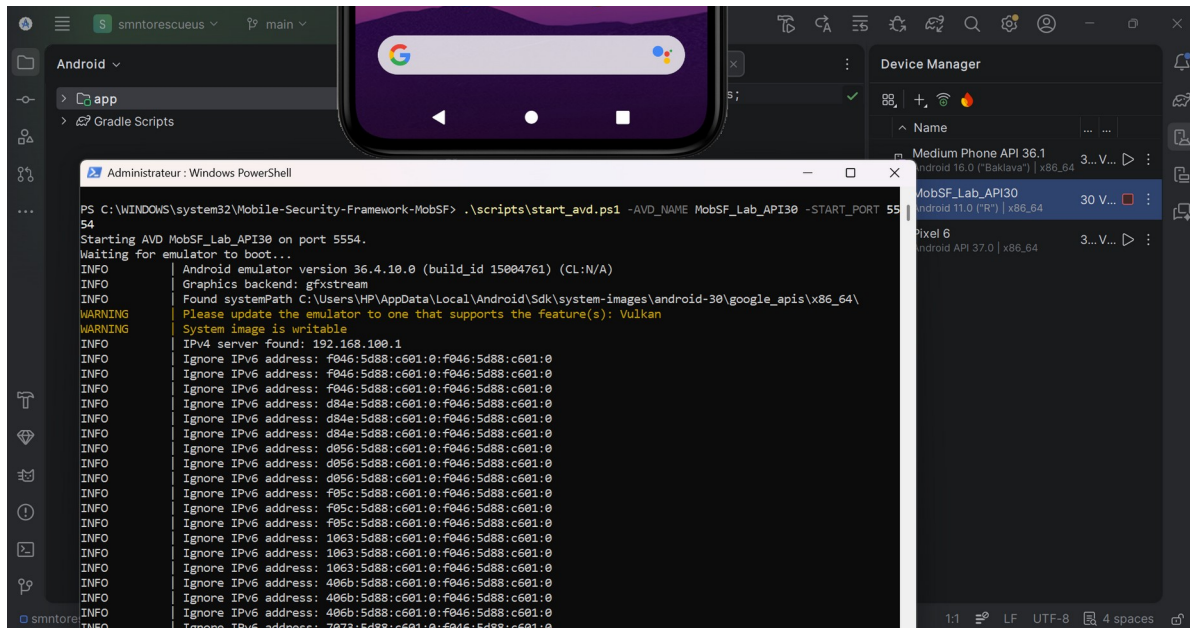


Figure 7 - Exécution du script de démarrage de l'AVD.

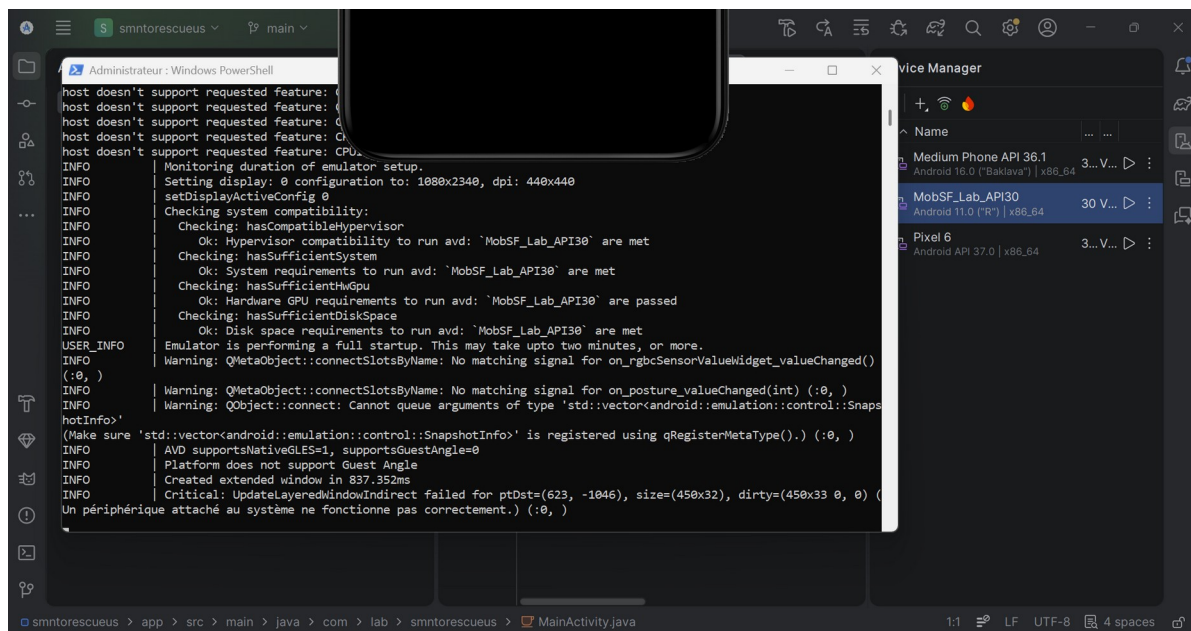


Figure 8 - Démarrage effectif de l'émulateur.

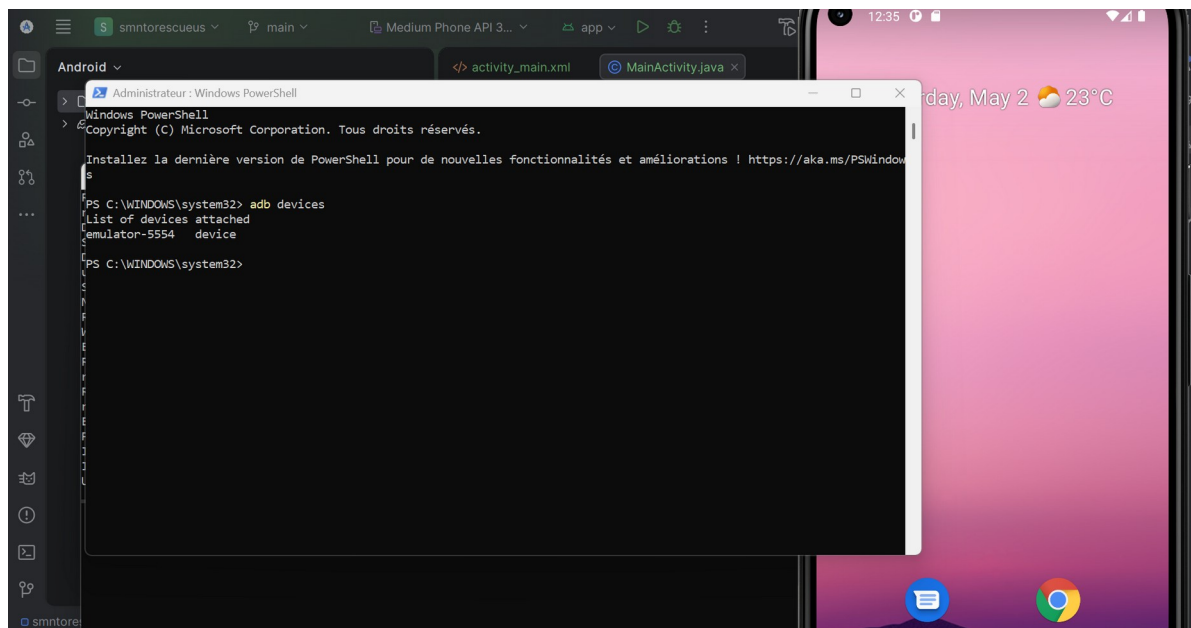


Figure 9 - Vérification ADB : l'émulateur est détecté.

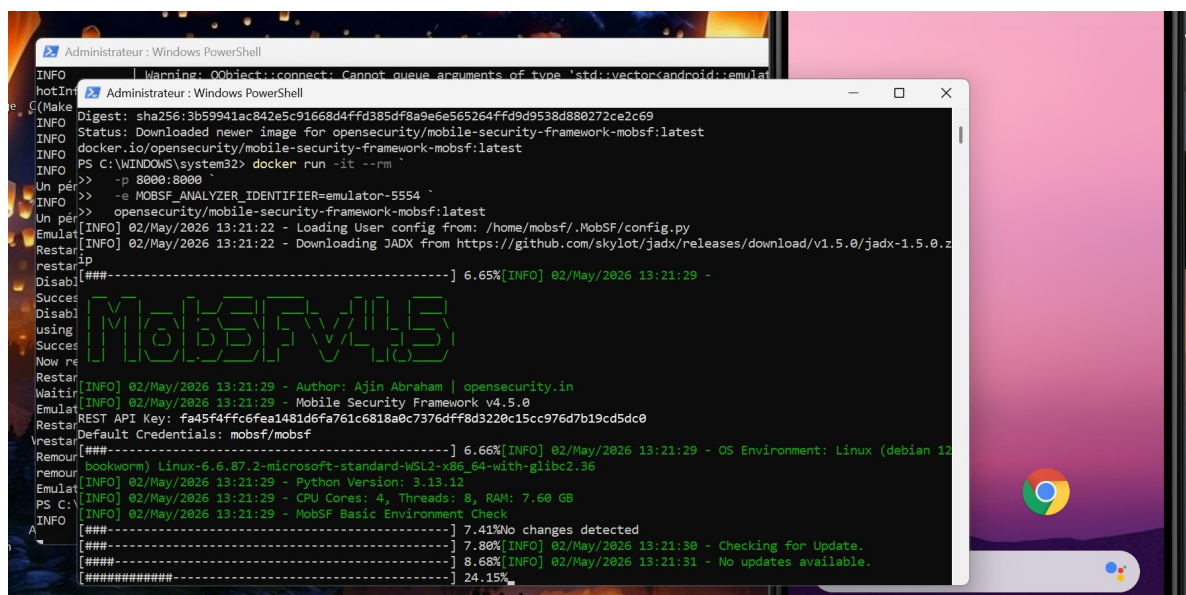


Figure 12 - Lancement du conteneur MobSF avec l'identifiant ADB.

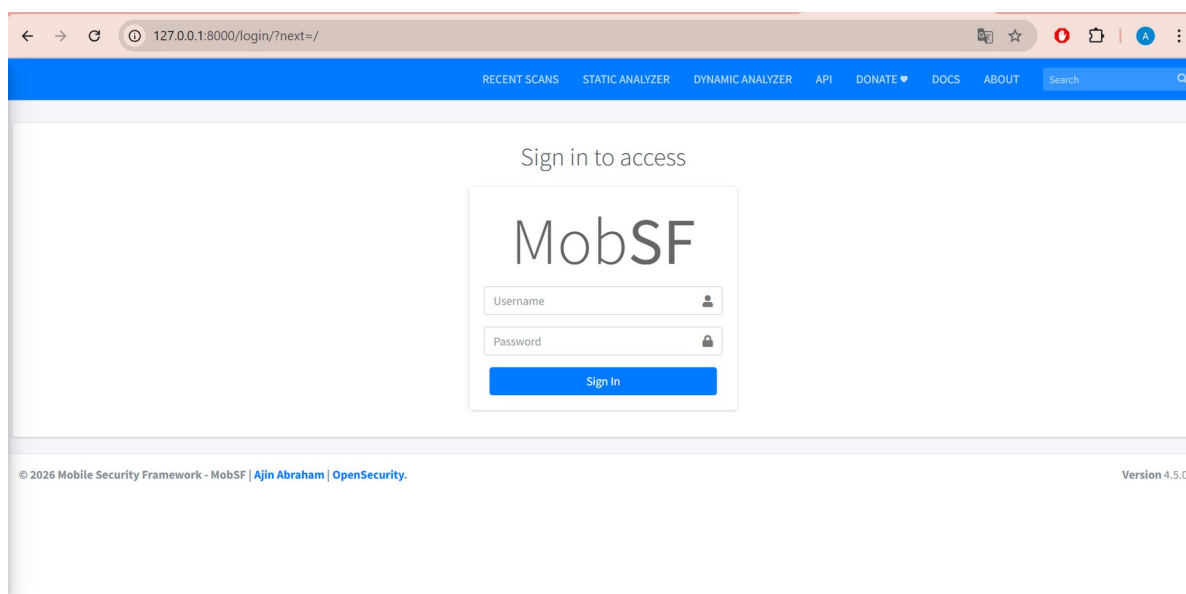


Figure 13 - Accès à l'interface web de MobSF.

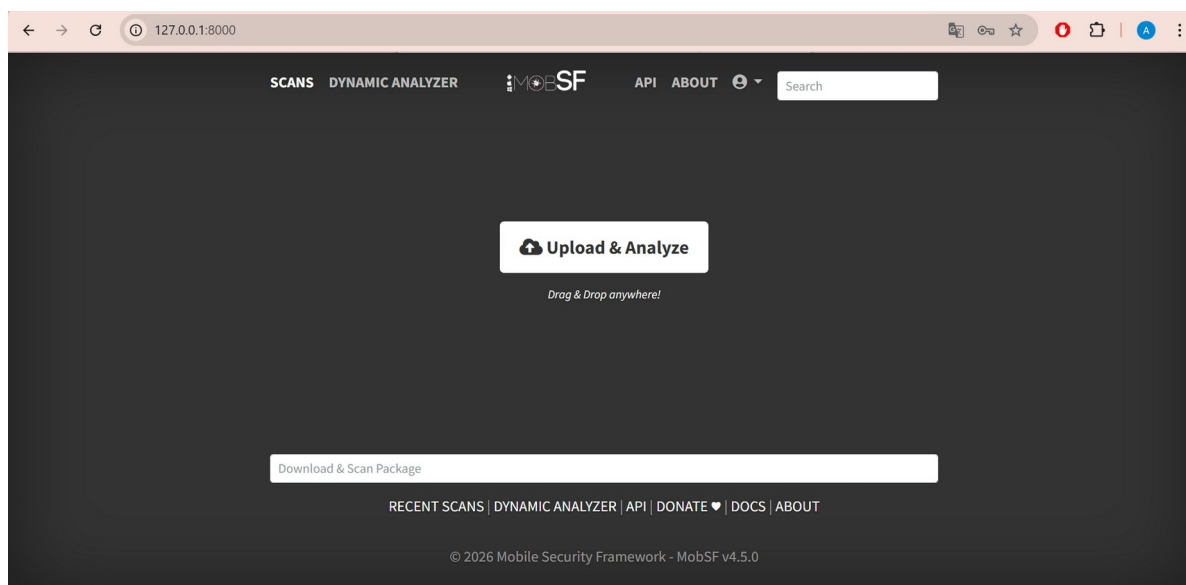


Figure 14 - Interface MobSF après connexion.

Les identifiants par défaut de MobSF sont utilisés pour accéder au tableau de bord, puis le service web est ouvert en local sur le port 8000.

5. Analyse statique de DIVA

Après le lancement de MobSF, l'APK DIVA est importée pour une première analyse statique. MobSF examine automatiquement le manifeste Android, les activités exportées, le code décompilé, les bibliothèques natives et les certificats de signature.

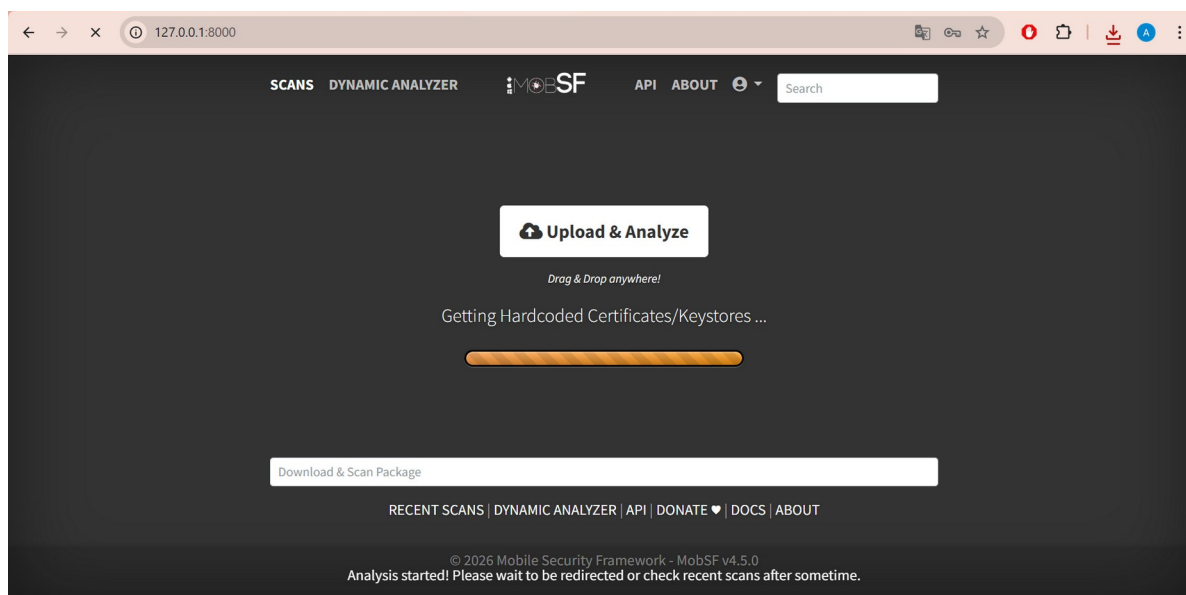


Figure 15 - Téléversement de l'APK DIVA dans MobSF.

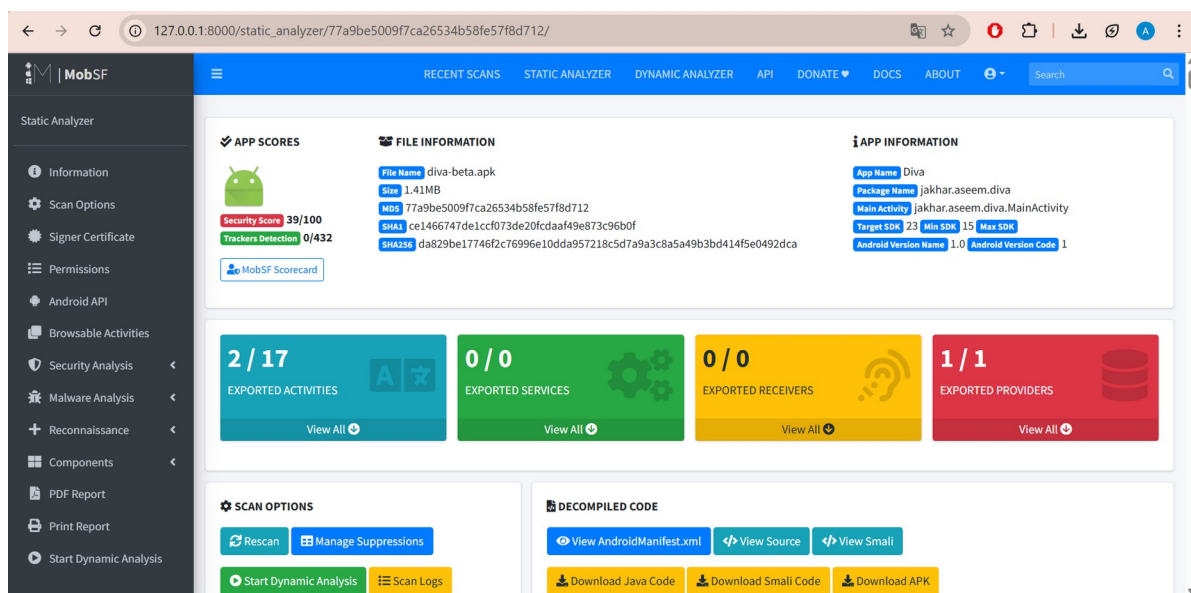


Figure 16 - Résultats initiaux de l'analyse statique.

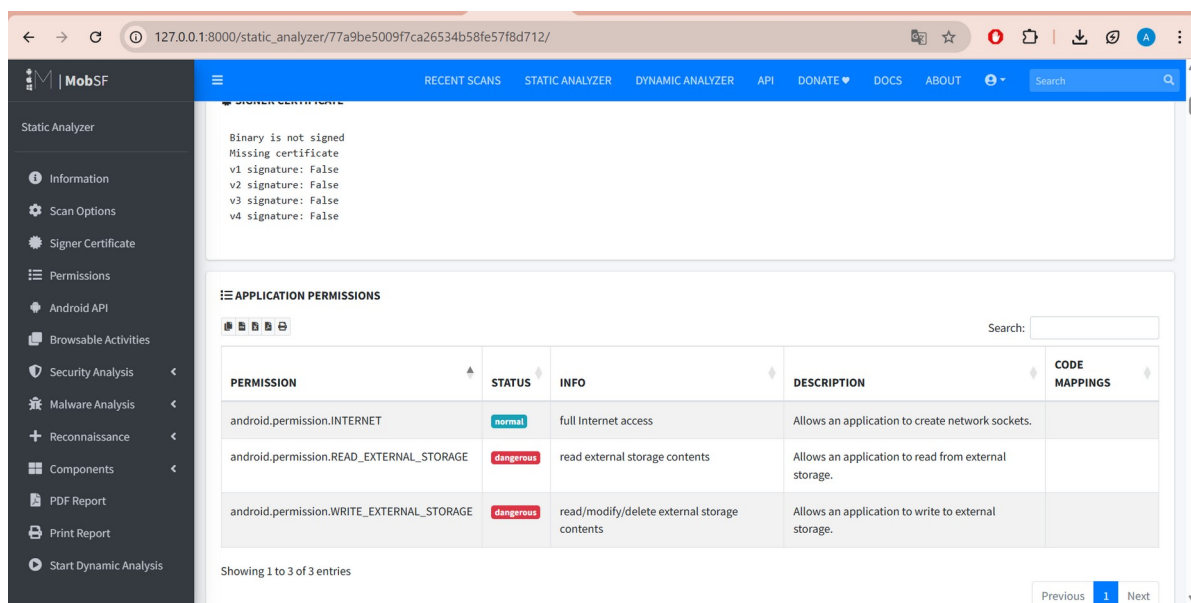
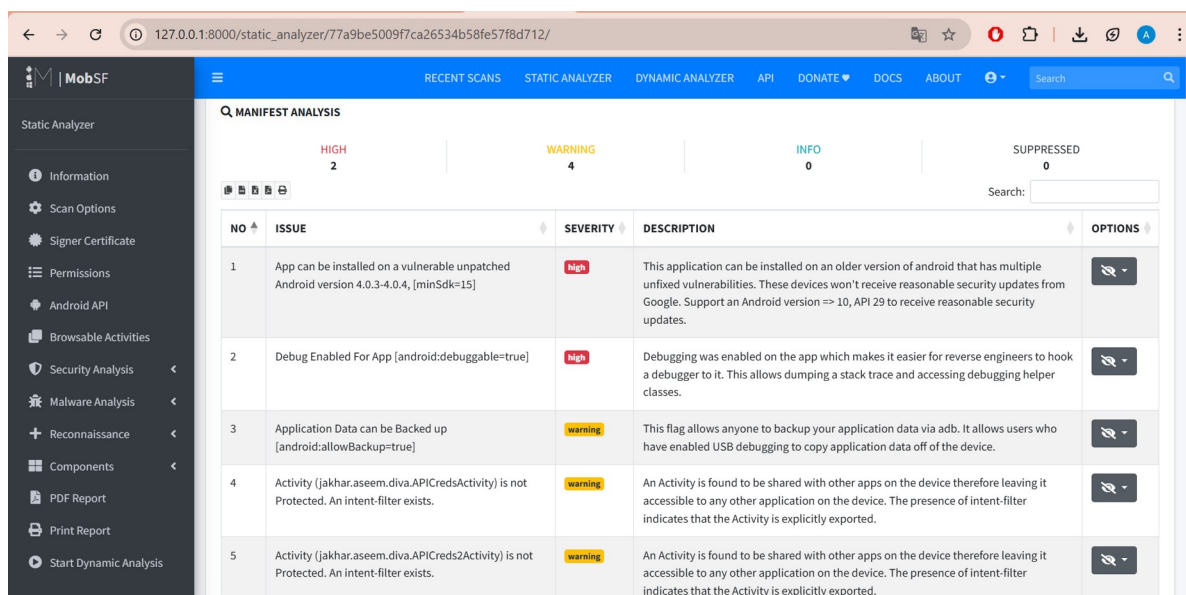


Figure 17 - Détails complémentaires du rapport statique.



NO	ISSUE	SEVERITY	DESCRIPTION	OPTIONS
1	App can be installed on a vulnerable unpatched Android version 4.0.3-4.0.4, [minSdk=15]	High	This application can be installed on an older version of android that has multiple unfixed vulnerabilities. These devices won't receive reasonable security updates from Google. Support an Android version => 10, API 29 to receive reasonable security updates.	[icon]
2	Debug Enabled For App [android:debuggable=true]	High	Debugging was enabled on the app which makes it easier for reverse engineers to hook a debugger to it. This allows dumping a stack trace and accessing debugging helper classes.	[icon]
3	Application Data can be Backed up [android:allowBackup=true]	Warning	This flag allows anyone to backup your application data via adb. It allows users who have enabled USB debugging to copy application data off of the device.	[icon]
4	Activity (jakhar.aseem.diva.APICredsActivity) is not Protected. An intent-filter exists.	Warning	An Activity is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device. The presence of intent-filter indicates that the Activity is explicitly exported.	[icon]
5	Activity (jakhar.aseem.diva.APICreds2Activity) is not Protected. An intent-filter exists.	Warning	An Activity is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device. The presence of intent-filter indicates that the Activity is explicitly exported.	[icon]

Figure 18 - Vue étendue des alertes et observations.

Cette étape met en évidence plusieurs indicateurs de risque typiques des applications mobiles vulnérables, comme des permissions sensibles, des composants exposés et des zones de stockage potentiellement non sécurisées.

6. Analyse dynamique

La partie dynamique permet de vérifier le comportement réel de l'application lors de son exécution. MobSF peut alors installer l'APK sur l'émulateur, configurer le proxy HTTPS, démarrer le support Frida et fournir une console d'observation en temps réel.

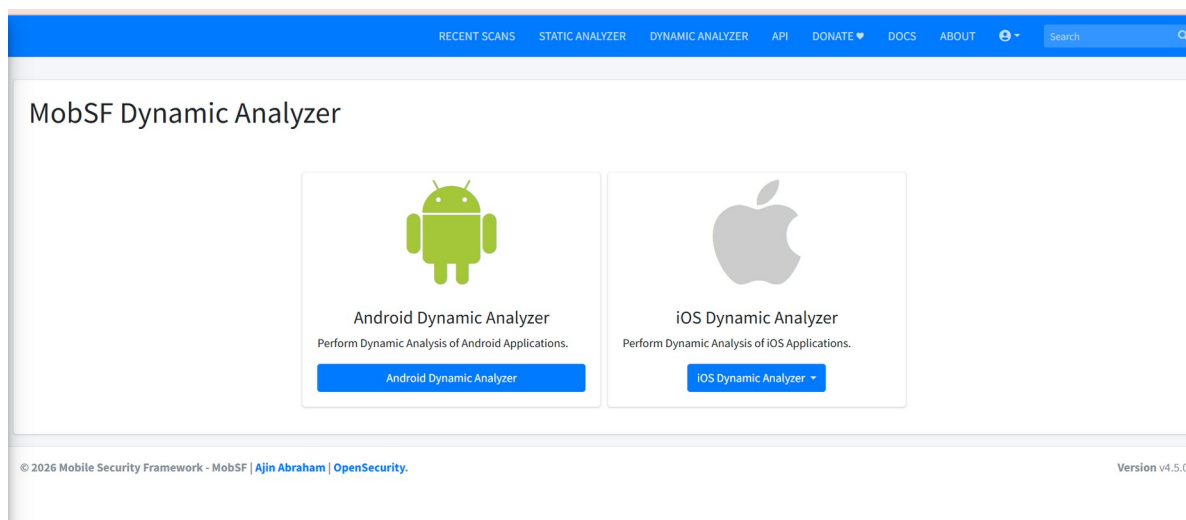


Figure 19 - Accès au module Dynamic Analyzer.

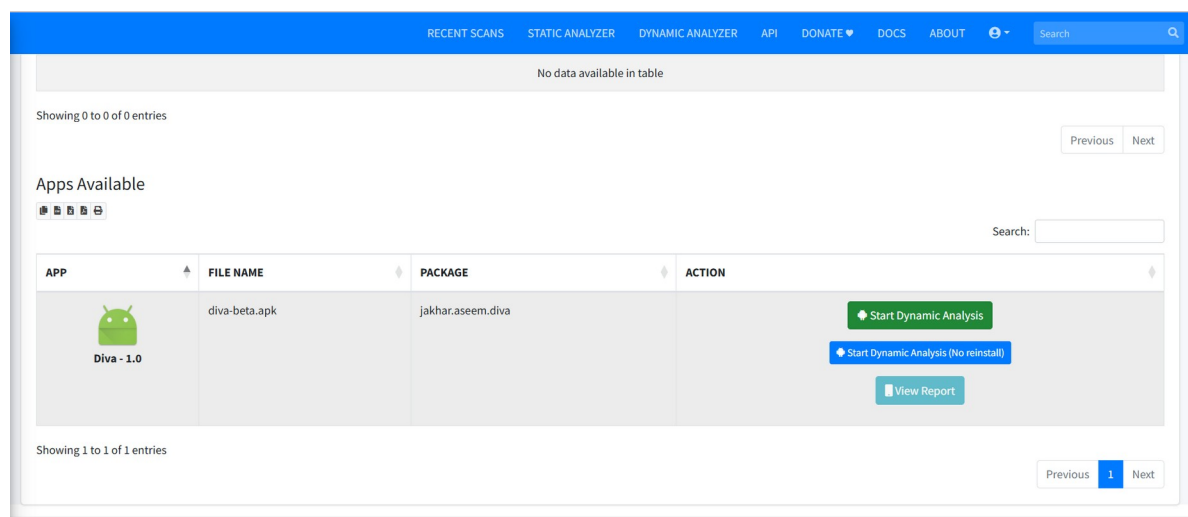


Figure 20 - Démarrage de la session d'analyse dynamique.

Pendant cette phase, l'analyste peut utiliser le stream logcat, tester les activités exportées, contrôler la sécurité TLS/SSL et générer des captures d'écran pour documenter les comportements observés.

7. Résultats et observations principales

- L'environnement a pu être mis en place correctement avec un AVD compatible et reconnu par ADB.
- MobSF a démarré en conteneur Docker et l'interface web a été accessible localement.
- L'APK DIVA a été chargée avec succès et analysée par MobSF en mode statique.
- La bascule vers l'analyse dynamique a permis de préparer l'exécution runtime de l'application.
- Le laboratoire confirme l'intérêt de MobSF pour regrouper en un seul flux l'analyse statique, le monitoring runtime et les tests de sécurité mobile.

En synthèse, l'application DIVA sert de support pédagogique pour comprendre les vulnérabilités mobiles les plus courantes et la manière dont MobSF les met en évidence.

8. Conclusion

Ce laboratoire a permis de mettre en œuvre une chaîne complète d'analyse dynamique Android dans un environnement Windows. La configuration de l'AVD, le déploiement de MobSF et l'étude de DIVA offrent une base solide pour reproduire d'autres tests de sécurité mobile, comparer différents comportements runtime et préparer des audits plus avancés.

Annexe - Synthèse des étapes

Étape	Contenu
1	Création de l'AVD sans Play Store
2	Clonage du dépôt MobSF
3	Démarrage de l'émulateur et vérification ADB
4	Lancement de MobSF via Docker
5	Téléversement et analyse statique de DIVA
6	Activation du Dynamic Analyzer